MockU - A one stop Mock Tests Platform Software Requirements Specification (SRS)

Project: MockU – A one stop Mock Tests Platform

Prepared by: Team Triumph
Date: 09/02/2026

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the MockU system. MockU is a mobile-based competitive exam mock test and performance guidance platform designed to help students prepare strategically for various competitive examinations. This document describes the functional and non-functional requirements, system constraints, user characteristics, and overall system behavior.

1.2 Scope

MockU is a cross-platform mobile application that enables students to attempt competitive exam mock tests and receive detailed performance analysis and guided feedback. It supports multiple examinations such as GATE, CAT, UPSC, JEE, and NEET, providing section-wise evaluation, identification of weak and strong areas, reference recommendations, and progress tracking dashboards. The system is intended for competitive exam aspirants and administrators managing test content, and it focuses primarily on mock testing and performance analysis rather than live instruction.

1.3 Definitions

- **Mock Test:** A practice exam that simulates the actual competitive exam pattern.

➔ **Section:** A specific division of an exam, such as Quantitative Aptitude or Reasoning.

➔ **Weak Area:** A topic or section where the student's performance falls below a set threshold.

➔ **Strong Area:** A topic or section where the student consistently performs well.

➔ **JWT (JSON Web Token):** A secure token used for authentication between client and server.

➔ **Rule-Based Analysis Engine:** A system that evaluates performance using predefined logical rules.

1.4 References

IEEE 830-1998 Standard for Software Requirements Specification

1.5 Overview

This SRS document is organized into sections including overall system description, functional requirements, non-functional requirements, system interfaces, database requirements, and future enhancements.

2. Overall Description

2.1 Product Perspective

MockU is a client-server based mobile application. The frontend is developed using React Native and communicates with a Node.js and Express.js backend through REST APIs. MongoDB is used for data storage, and authentication is implemented using JWT-based secure token validation.

2.2 Product Functions

The major functions of the MockU system include:

- User registration and login.
- Attempting exam-specific mock tests.
- Automatic evaluation of answers.
- Section-wise and topic-wise performance analysis.

- Identification of weak and strong areas.
- Recommendation of reference books and preparation strategies.
- Display of dashboard statistics and progress trends.
- Administrative management of questions and tests.

2.3 User Characteristics

The system will have two main types of users:

Students

Students preparing for competitive examinations with basic smartphone usage knowledge.

Administrators

Authorized users responsible for managing exams, questions, and monitoring system data.

2.4 Constraints

- Internet connectivity is required for test attempt and data synchronization.
- Mobile device must support Android or iOS platforms.
- Backend server must be continuously available.
- JWT authentication must be implemented for secure access.

2.5 Assumptions and Dependencies

- Users have basic knowledge of competitive exams.
- Questions are pre-uploaded by administrators.
- System depends on stable backend hosting and MongoDB database environment.

3. Specific Requirements

3.1 Functional Requirements

FR1: User Registration

The system shall allow users to register using email and password.

FR2: User Authentication

The system shall authenticate users using JWT-based authentication.

FR3: Secure Password Storage

The system shall store passwords in hashed format.

FR4: Exam Selection

The system shall allow users to select an exam category.

FR5: Mock Test Attempt

The system shall allow users to attempt mock tests with time limits.

FR6: Auto Submission

The system shall automatically submit the test when the timer expires.

FR7: Score Calculation

The system shall calculate total score and section-wise scores.

FR8: Accuracy Calculation

The system shall compute accuracy percentage.

FR9: Negative Marking

The system shall apply negative marking as per exam rules.

FR10: Performance Analysis

The system shall identify weak & strong areas based on performance thresholds.

FR11: Reference Recommendation

The system shall suggest reference books for weak topics.

FR12: Do’s and Don’ts Guidance

The system shall provide exam-specific preparation tips.

FR13: Progress Tracking

The system shall track user performance across multiple tests.

FR14: Dashboard Display

The system shall display average score, total tests, best score, and weak areas.

FR15: Mistake Review

The system shall allow users to review incorrect answers.

FR16: Admin Question Management

The system shall allow administrators to add, update, and delete questions.

FR17: Admin Test Creation

The system shall allow administrators to create mock tests.

3.2. Non-Functional Requirements

NFR1: Performance

System response time shall not exceed 2 seconds for API requests.

NFR2: Security

All API endpoints shall require JWT validation.

NFR3: Scalability & Usability

System shall support multiple concurrent users and Mobile interface shall be user-friendly and intuitive.

NFR4: Reliability

User data and test results shall not be lost.

NFR5: Availability

System shall maintain high availability during test periods.

3.3. External Interfaces

User Interface

Mobile-based interface built using React Native.

Backend Interface

REST APIs built using Node.js and Express.js.

Database Interface

MongoDB used for storing user data, questions, tests, and results.

4. Database Requirements

The system shall maintain the following collections in MongoDB:

Users Collection

- userId
- name
- email
- password (hashed)
- exam preferences

Questions Collection

- questionId
- examType
- section
- difficulty
- options
- correctAnswer

Results Collection

- userId
- testId
- score
- accuracy
- sectionScores
- timestamp

5. System Architecture

MockU follows a client-server architecture where:

- React Native mobile application acts as client.

- Node.js and Express.js act as backend server.
- MongoDB stores application data.
- JWT tokens ensure secure authentication.
- Communication occurs through REST APIs over HTTP.

6. Future Enhancements

- Adaptive difficulty mock tests.
- AI-based personalized study planner.
- Rank prediction module.
- Mobile push notifications.
- Offline test attempt mode.
- Cloud deployment for scalability.

7. Document Approval

Prepared by: Team Triumph
Verified by: Ms. Susmitha E
Institution: RGUKT RK VALLEY

8. Change Management Process

Version	Date	Change Description	Author
1.0	09/02/2026	Initial SRS Created	Team Triumph